Iteration I — `for` and `range`

Lecture 22

CS 1001 Kirk Duran

Today

- Explain why repeated computation needs a new control-flow structure.
- Trace a `for` loop one iteration at a time.
- Treat the loop variable as a name that is repeatedly rebound.
- Use `range` to describe integer sequences for iteration.
- Distinguish direct iteration over values from iteration over indexes.
- Introduce an **accumulator** as state updated across iterations.

Key idea

A loop does not execute "all at once." It repeatedly binds a loop variable, executes the body, and then advances to the next value.

Part 1: Why Programs Need Iteration

Repeated Statements Do Not Scale

Suppose we want to display four values from a list.

Without iteration

```
print(values[0])
print(values[1])
print(values[2])
print(values[3])
```

- The same operation is written repeatedly.
- The code is tied to one specific list length.
- A larger input requires more source code.

Key idea

Iteration lets the *program* repeat a computation instead of requiring the programmer to repeat source code.

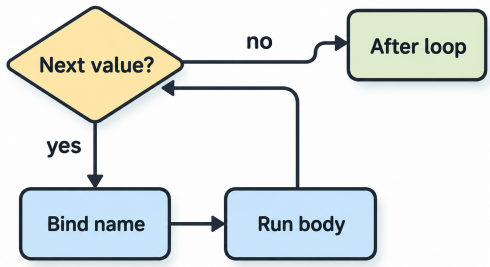
Definition: Iteration

Iteration is repeated execution of a block of code according to a control structure.

For today's `for` loops:

1. Get the next supplied value, if one exists.
2. Bind the loop variable to it.
3. Execute the body, then request the next value.

When no values remain, continue after the loop.



Expressions, assignments, calls, and conditionals retain their usual meanings.

Anatomy of a `for` Loop

General form

```
for variable in sequence:
   statement
   statement
```

Part	Role
`for`	begins the loop
loop variable	receives one value at a time
sequence	supplies values to the loop
indented body	executes once for each supplied value

The Execution Model

Consider:

Code

```
for value in [4, 9, 2]:
   print(value)
```

Python conceptually performs:

1. bind `value` to 4; execute the body;
2. bind `value` to 9; execute the body;
3. bind `value` to 2; execute the body;
4. no values remain, so execution continues after the loop.

Key idea

One loop body can execute many times, but each individual execution still follows ordinary statement-by-statement evaluation.

Trace a Simple Loop

Code

```
values = [4, 9, 2]
for value in values:
   answer = value + 1
   print(answer)
```

In class

Before revealing the trace, predict the three values printed and the sequence of bindings for `value` and `answer`.

Trace Table

Iteration	`value`	`value + 1`	`answer`
1	4	5	5
2	9	10	10
3	2	3	3

Printed output

```
5
10
3
```

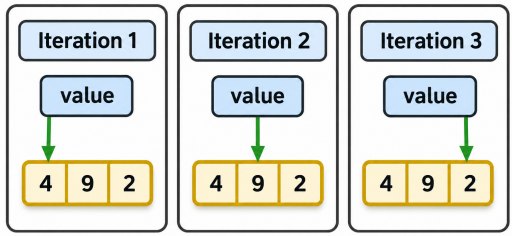
The body is evaluated three separate times using three different bindings for `value`.

The Loop Variable Is a Normal Binding

Code

```
for value in [4, 9, 2]:
  print(value)
```

The picture shows three moments in time. Each supplied element takes over the `value` binding; the list stays unchanged.



Key idea

Each iteration **rebinds** the loop variable to the next supplied value. It does not replace a position in the list.

Indentation Defines the Loop Body

Inside the loop

```
for x in [1, 2, 3]:
    print(x)
    print("again")
```

Both prints execute three times.

After the loop

```
for x in [1, 2, 3]:
    print(x)
print("done")
```

`"done"` prints once.

Tracing rule

Before tracing a loop, mark exactly which statements belong to its indented body.

Part 2: Iterating Directly Over Sequential Data

Direct Iteration Over Values

Lists already contain the values we often want to process.

Code

```
temperatures = [68, 71, 74]
for temperature in temperatures:
   print(temperature)
```

The loop variable receives the list elements directly:

$$68 \rightarrow 71 \rightarrow 74$$

Key idea

If a computation needs the values themselves, direct iteration usually expresses that intent most clearly.

Prediction: Expressions Inside a Loop

Code

```
values = [2, 5, 8]
for value in values:
   result = value * 2
   print(result)
print("done")
```

In class

Predict the complete output. How many times is `value * 2` evaluated? How many times is `"done"` printed?

Prediction: Reveal

Output

```
4
10
16
done
```

- The loop body executes once for each of the three list elements.
- `result` is rebound during each iteration.
- The final print is outside the loop, so it executes once.

Rebinding the Loop Variable Does Not Mutate the List

Code

```
values = [2, 5, 8]
for value in values:
   value = value * 2
print(values)
```

Output: `[2, 5, 8]` Each multiplication rebinds `value` to a new integer. The next iteration then binds it to the next original element.

Key idea

As in Lecture 21, rebinding a name differs from mutating a list. This code never assigns to `values[index]`.

The Same Mechanism Applies to Other Sequences

A `for` loop can receive values from sequential data other than lists.

Example

```
for character in "cat":
   print(character)
```

The bindings are:

`character` → `"c"` `"a"` `"t"`

For now

Today the emphasis is the loop execution model. We will use iteration over lists and strings more systematically in the next lecture.

A Loop Inside a Function

Code

```
def display(values:  list[int]) -> None:
   for value in values:
      print(value)
```

Function-call reasoning still applies:

1. evaluate the argument;
2. bind the parameter `values`;
3. execute the body: the loop repeatedly binds `value` and prints it;
4. the call ends after the loop completes.

Key idea

Iteration happens *inside* the existing call frame; it does not create a new function call for each iteration.

Part 3: Integer Sequences with `range`

Why `range`?

Sometimes we want integer values that are not already stored in a list.

Example goal

Repeat a computation for the integers 0, 1, 2, and 3.

Python's `range` describes a sequence of integers suitable for iteration.

Code

```
for i in range(4):
   print(i)
```

range(stop)

General form

`range(stop)`

It begins at `0` and stops **before** `stop`.

Expression	Integer values supplied
`range(4)`	`0, 1, 2, 3`
`range(1)`	`0`
`range(6)`	`0, 1, 2, 3, 4, 5`

The Stop Value Is Exclusive

You have already seen this convention in slicing.

`values[1:4]`	indexes `1, 2, 3`
`range(1, 4)`	integers `1, 2, 3`

Key idea

In both slicing and `range`, the start is included and the stop is excluded.

This makes `range(len(values))` naturally line up with valid list indexes from `0` through `len(values) - 1`.

range(start, stop)

General form

```
range(start, stop)
```

Expression	Integer values supplied
`range(2, 6)`	`2, 3, 4, 5`
`range(5, 8)`	`5, 6, 7`
`range(3, 4)`	`3`

The default step is 1.

range(start, stop, step)

General form

`range(start, stop, step)`

Expression	Integer values supplied
`range(0, 8, 2)`	`0, 2, 4, 6`
`range(1, 10, 3)`	`1, 4, 7`
`range(2, 9, 2)`	`2, 4, 6, 8`

Key idea

The step controls how the current integer changes before the next iteration.

Negative Steps

A negative step describes decreasing integer values.

Expression	Integer values supplied
`range(4, 0, -1)`	`4, 3, 2, 1`
`range(6, 1, -2)`	`6, 4, 2`

The stop remains exclusive even when the sequence decreases.

A Valid `range` Can Be Empty

Expression	Why it supplies no values
`range(0)`	Start equals stop.
`range(3, 3)`	Start equals stop.
`range(5, 2)`	Positive step, but start is already past stop.
`range(1, 5, -1)`	Negative step moves away from stop.

An empty sequence means the body executes **zero times**; execution continues after the loop.

Different case: an invalid step

`range(0, 5, 0)` raises `ValueError`. A zero step is invalid, not an empty range.

Predict the Values from `range`

Without running Python, write the values supplied by each expression:

1. `range(5)`
2. `range(2, 7)`
3. `range(1, 8, 2)`
4. `range(5, 1, -1)`

In class

For each one, identify the start, stop, and step before listing the values.

`range` Practice: Reveal

Expression	Start	Stop	Step	Values supplied
`range(5)`	0	5	1	`0, 1, 2, 3, 4`
`range(2, 7)`	2	7	1	`2, 3, 4, 5, 6`
`range(1, 8, 2)`	1	8	2	`1, 3, 5, 7`
`range(5, 1, -1)`	5	1	-1	`5, 4, 3, 2`

Key idea

Identify the defaults first. Stop remains exclusive for both increasing and decreasing progressions.

Trace a Loop Driven by `range`

Code

```
for i in range(1, 5):
   value = i * 10
   print(value)
```

Iteration	i	value
1	1	10
2	2	20
3	3	30
4	4	40

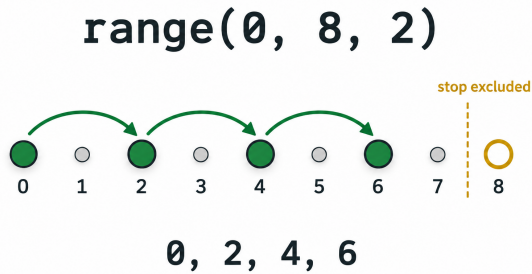
A `range` Describes the Integers

Code

```
numbers = range(0, 8, 2)
```

It supplies `0, 2, 4, 6`.

- Start at `0`.
- Advance by `2`.
- Stop before `8`.



Key idea

A `range` describes an integer progression without constructing a list of all its values.

Using `range(len(...))` for Indexes

Sometimes the computation needs an element's **index**.

Code

```
values = [8, 3, 6]
for idx in range(len(values)):
   print(idx, values[idx])
```

Since `len(values)` is 3:

$$\texttt{range(3)} \Rightarrow \texttt{0, 1, 2}$$

Those are exactly the valid indexes for the list.

Values or Indexes?

Need each value

```
for value in values:
  print(value)
```

Loop variable: **element value**

Need each index

```
for idx in range(len(values)):
  print(idx)
```

Loop variable: **integer index**

Key idea

Do not introduce indexes unless the computation actually needs the position. Direct iteration is simpler when only the values matter.

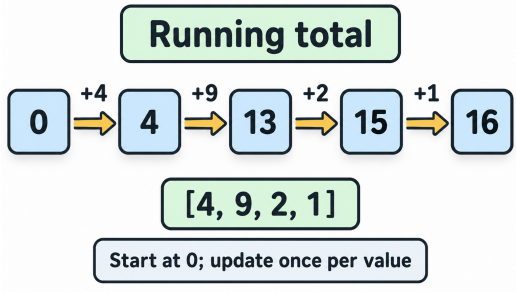
Part 4: State That Changes Across Iterations

Some Computations Need Memory Across Iterations

Each iteration adds one value from `[4, 9, 2, 1]` to the sum so far.

Required state

`total`: the sum of the values processed so far.



Key idea

The boxes show successive values of **one** variable, not separate accumulators. Each update uses the previous total.

Definition: Accumulator

Definition

An **accumulator** is a variable that stores a partial result and is updated repeatedly as a loop executes.

A common structure is:

Pattern

initialize accumulator
`for value in sequence:`
    update accumulator using `value`
use the completed accumulator after the loop

Today we use summation only to understand the mechanics. We will study additional loop patterns next lecture.

The Lab Example: `tally`

Code

```
def tally(nums:  list[int]) -> int:
   total = 0
   for num in nums:
      total = total + num
   return total
```

For the call:

```
tally([4, 9, 2, 1])
```

In class

Predict the values bound to `num` and the value of `total` after each iteration.

Tracing the Accumulator

Iteration	`num`	`total` **before**	`total` **after**
–	–	–	0
1	4	0	4
2	9	4	13
3	2	13	15
4	1	15	16

Key idea

The next iteration begins with the accumulator value produced by the previous iteration.

Initialization Determines the Starting State

Why does `tally` begin with:

```
total = 0
```

Before any elements have been processed, the sum-so-far should be zero.

- initialization happens once, before the loop;
- updating happens once per iteration;
- the final value is used after all iterations complete.

Tracing discipline

Record the initial value before tracing. With an empty input, `tally([])` returns `0`: no updates occur.

Return Placement Matters

Both fragments are inside a function, after `total = 0`.

After the loop

```
for num in nums:
    total = total + num
return total
```

All elements are processed before returning.

Inside the loop

```
for num in nums:
    total = total + num
    return total
```

For nonempty input, the first iteration returns immediately. Empty input reaches the function end and returns `None`.

Key idea

`return` keeps its old meaning: it immediately ends the current function call, even when it appears inside a loop.

Part 5: Integrating the Model

Prediction Challenge

Code

```
def compute(start:  int, stop:  int) -> int:
   total = 0
   for value in range(start, stop):
      total = total + value
   return total

answer = compute(2, 6)
```

In class

Determine the values successively bound to `value`, the value of `total` after each iteration, and the final value bound to `answer`.

Prediction Challenge: Reveal

For `compute(2, 6)`, the loop receives `2, 3, 4, 5`.

Iteration	`value`	`total` before	`total` after
1	2	0	2
2	3	2	5
3	4	5	9
4	5	9	14

After the loop

`return total` returns 14, so `answer` is bound to 14.

In class

What would `compute(6, 2)` return? Explain using the range and the initial total.

Where We Stop Today

Today's responsibility is the mechanics of iteration:

- how a `for` loop repeatedly executes;
- how the loop variable is rebound;
- how `range` supplies integer values;
- when to iterate over values versus indexes;
- how one accumulator can preserve state across iterations.

Next lectures

Lecture 23: common loop patterns such as counting, building results, and combining loops with conditionals.

Lecture 24: higher-level sequence patterns, including comprehensions and map/filter-style reasoning.

Takeaways

1. A `for` loop processes one supplied value at a time.
2. The loop variable is repeatedly rebound to the next value.
3. The indented body executes once per supplied value.
4. `range` describes integer values using start, exclusive stop, and optional step.
5. Use direct iteration when you need values; use indexes when you need positions.
6. An accumulator stores a partial result across iterations.
7. Existing rules still apply: expressions evaluate normally, assignment rebinds names, and `return` immediately ends a function call.